



Restaurant, Order & Delivery Management System

Team 5 | DBMS Course Project

RDBMS MySQL 8.x

Language SQL

Course DBMS Project

Design ER Model

Normalization Analysis

Repository GitHub

FoodGo is a **MySQL-based relational database** designed to model the core operations of a food-delivery platform. The system connects **customers, delivery addresses, restaurants, menu items, orders, delivery partners, deliveries, payments and reviews** in a structured relational model.

The project focuses on **ER modeling, relational schema design, functional dependencies, normalization, integrity constraints, realistic sample data and SQL-based business analysis**.

[!NOTE] **Normalization note:** The project includes a dedicated, **read-only** `normalization.sql` verification script. It demonstrates 1NF and 2NF, analyzes the dependencies relevant to 3NF, verifies lossless reconstruction, and checks price, total and cross-table consistency. Under the stated business dependency `address_id → customer_id`, the current physical `orders` table retains a transitive dependency, so the project does **not** claim that the entire current physical schema is strictly 3NF. The script documents this explicitly rather than silently changing the supplied schema.

Table of Contents

#	Section	#	Section
1	Team Members	10	Integrity Constraints
2	Project Overview	11	Sample Data
3	Project Objectives	12	Normalization Verification Queries
4	System Scope	13	Business Questions
5	Technology Stack	14	Key Design Decisions
6	Repository Structure	15	SQL Concepts Demonstrated
7	Database Design	16	How to Run the Project
8	Relationships	17	Project Workflow
9	Normalization	18	Conclusion

Team Members

S.No.	Student Name	USN
1	Rehaan Saha	AU25UG-046
2	S Thanmayee	AU25UG-049
3	Samhita Bhuvanagiri	AU25UG-050
4	Sarang Siva Thilakan	AU25UG-054
5	Shahiba Arshiya Banu	AU25UG-055
6	Supriya R	AU25UG-058

Project Overview

A food-delivery platform coordinates three major participants:

Participant	Role
Customers	Place orders and maintain delivery addresses
Restaurants	Provide menus and prepare orders
Delivery Partners	Handle order delivery

FoodGo brings these activities together in one relational database. It stores customer information, saved addresses, restaurants, menu items, orders, order-line details, delivery assignments, payments and reviews.

The database was designed from the project requirements and the suggested FoodGo database specification, then implemented using MySQL.

Project Objectives

The main objectives are to:

- Design a realistic relational database for a food-delivery system.
- Identify entities, attributes, keys and relationships.
- Create an ER diagram with cardinalities and participation information.
- Convert the ER model into relational tables.
- Apply primary keys and foreign keys.
- Maintain data integrity using `NOT NULL`, `UNIQUE`, `CHECK`, `DEFAULT` and referential actions.
- Identify important functional dependencies.
- Demonstrate normalization through 1NF, 2NF and 3NF analysis.
- Insert realistic, referentially consistent sample data.
- Write SQL queries for practical business analysis.
- Verify important cross-table business rules.
- Maintain the complete project in a reproducible GitHub repository.

System Scope

```
<table> <tr> <td width="50%" valign="top">
```

Customer Management

- Store customer profiles.
- Maintain unique phone numbers and emails.
- Manage multiple saved delivery addresses.

Restaurant Management

- Store restaurant information.
- Maintain cuisine and city details.
- Track whether a restaurant is active.
- Store menu items and their current catalog prices.

Order Management

- Record customer orders.
- Associate orders with restaurants and delivery addresses.
- Store order status and order time.
- Store the total amount.
- Record the individual items included in each order.

</td> <td width="50%" valign="top">

Delivery Management

- Maintain delivery partner details.
- Assign delivery partners to orders.
- Record pickup and delivery timestamps.
- Allow **at most one** delivery record per order.

Payment Management

- Store payment amount and method.
- Track payment status.
- Allow **at most one** payment record per order in the current schema.

Review Management

- Store restaurant and delivery ratings.
- Store customer comments.
- Allow **at most one** review record per order in the current schema.

</td> </tr> </table>

Technology Stack

Layer	Technology
RDBMS	RDBMSMySQL 8.x
Query Language	SQL
Schema / DDL	MySQL DDL
Data / DML	MySQL INSERT statements
Design	ER Diagram / Relational Schema
Documentation	Markdown
Repository	RepositoryGitHub

Repository Structure

 Path	 File	 Purpose
/	README.md	Project documentation
schema/	create_tables.sql	Database, tables, keys and constraints
data/	insert_data.sql	Realistic sample data
queries/	queries.sql	Business and analytical SQL queries
normalization/	normalization.sql	Read-only 1NF, 2NF, 3NF analysis and verification
diagrams/	ER_Diagram.png	Entity-Relationship diagram
diagrams/	Relational_Schema.png	Relational schema diagram
docs/	Report.pdf	Project report
docs/	Design_Rationale.md	Design decisions and rationale

text

```
FoodGo/
├──  README.md
├──  schema/
│   └── create_tables.sql
├──  data/
│   └── insert_data.sql
├──  queries/
│   └── queries.sql
├──  normalization/
│   └── normalization.sql
├──  diagrams/
│   ├── ER_Diagram.png
│   └── Relational_Schema.png
└──  docs/
    ├── Report.pdf
    └── Design_Rationale.md
```

[!TIP] Rename the files/folders above to match the exact names in the repository if your GitHub folder names are different.

Database Design

FoodGo contains the following **10 core tables**:

#	Table	Purpose
1	 <code>customers</code>	Stores customer profiles and contact information
2	 <code>addresses</code>	Stores saved delivery addresses belonging to customers
3	 <code>restaurants</code>	Stores restaurant information
4	 <code>menu_items</code>	Stores dishes offered by restaurants
5	 <code>delivery_partners</code>	Stores delivery partner information
6	 <code>orders</code>	Stores order-level information
7	 <code>order_items</code>	Stores individual items within orders
8	 <code>deliveries</code>	Stores delivery assignment and timing
9	 <code>payments</code>	Stores payment information
10	 <code>reviews</code>	Stores restaurant/delivery ratings and comments

Entity-Relationship Overview

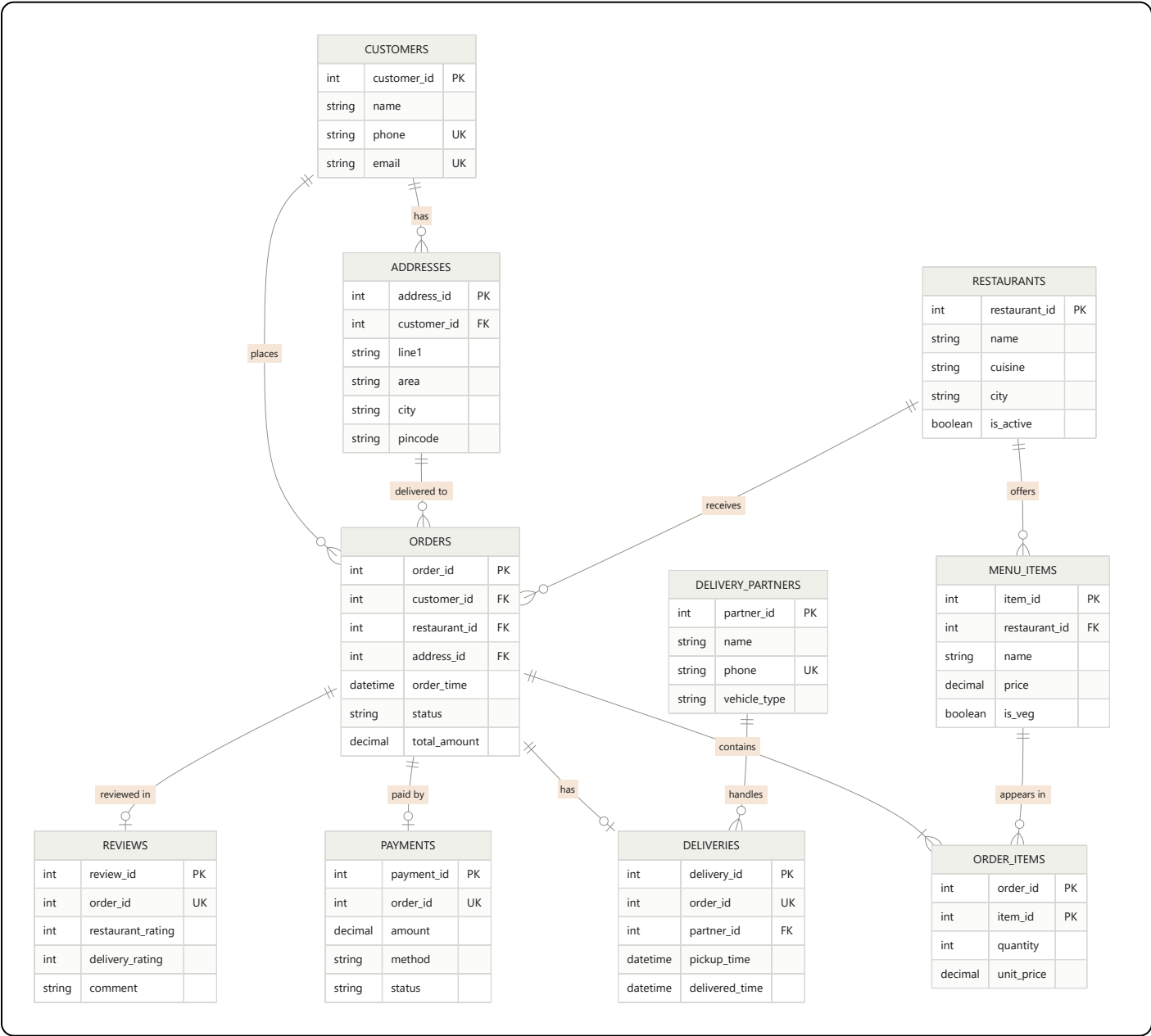


Table Attributes and Keys

<details> <summary>👤 customers</summary>

Attribute

Description

customer_id

🔑 Primary key

name

Customer name

phone

Unique contact number

email

Unique email address

Important constraints: `customer_id` is the primary key; `phone` and `email` have unique constraints.

addresses

Attribute	Description
<code>address_id</code>	 Primary key
<code>customer_id</code>	 Foreign key referencing <code>customers</code>
<code>line1</code>	Address line
<code>area</code>	Area/locality
<code>city</code>	City
<code>pincode</code>	Postal code

Relationship: One customer can have many addresses.

restaurants

Attribute	Description
<code>restaurant_id</code>	 Primary key
<code>name</code>	Restaurant name
<code>cuisine</code>	Cuisine type
<code>city</code>	Restaurant location
<code>is_active</code>	Indicates whether the restaurant is active

menu_items

Attribute	Description
item_id	🔑 Primary key
restaurant_id	🔗 Foreign key referencing restaurants
name	Food item name
price	Current catalog price
is_veg	Vegetarian/non-vegetarian indicator

Relationship: One restaurant can offer many menu items.

</details> <details> <summary> 🚚 delivery_partners</summary>

Attribute	Description
partner_id	🔑 Primary key
name	Delivery partner name
phone	Unique contact number
vehicle_type	Bike, scooter, bicycle or car

</details> <details> <summary> 📄 orders</summary>

Attribute	Description
order_id	🔑 Primary key
customer_id	🔗 Foreign key referencing customers
restaurant_id	🔗 Foreign key referencing restaurants
address_id	🔗 Foreign key referencing addresses
order_time	Time when order was placed
status	Order lifecycle status
total_amount	Stored order total

Allowed order statuses:

placed → preparing → out_for_delivery → delivered | cancelled
</details> <details> <summary>🛒 order_items</summary>

Attribute	Description
order_id	🔑 Part of composite primary key; 🔗 FK to orders
item_id	🔑 Part of composite primary key; 🔗 FK to menu_items
quantity	Number of units ordered
unit_price	Price captured for the item at order time

The primary key is:

```
sql
PRIMARY KEY (order_id, item_id)
```

This table resolves the **many-to-many** relationship between orders and menu items.

</details> <details> <summary>🚚 deliveries</summary>

Attribute	Description
delivery_id	🔑 Primary key
order_id	Unique 🔗 foreign key to orders
partner_id	🔗 Foreign key to delivery_partners
pickup_time	Pickup timestamp
delivered_time	Delivery completion timestamp

The unique constraint on order_id means an order can have **at most one** delivery record.

</details> <details> <summary>💵 payments</summary>

Attribute	Description
payment_id	🔑 Primary key
order_id	Unique 🔗 foreign key to orders
amount	Payment amount
method	Card, UPI, cash or netbanking
status	Pending, completed, failed or refunded

The current schema permits **at most one** payment record per order. The foreign key/unique constraint does not by itself force every order to have a payment record.

</details> <details> <summary> ⭐ reviews</summary>

Attribute	Description
review_id	🔑 Primary key
order_id	Unique 🔗 foreign key to orders
restaurant_rating	Rating from 1 to 5
delivery_rating	Rating from 1 to 5
comment	Customer feedback

The current schema permits **at most one** review per order.

</details>

 `create_tables.sql` — Version 2 Changes

In version 2 of our `create_tables.sql` file, we made some improvements **without removing any existing functionality**. The file increased from **198 lines to 284 lines**.

Change

Details

3 new `CHECK` constraints

Validate the format of important fields: customer phone numbers must contain 10 digits, address pincodes must contain 6 digits, and delivery partner phone numbers must contain 10 digits

2 new indexes on `orders`

One for order `status` and another for `order_time`, improving the efficiency of queries that frequently search or sort orders using these columns

3 new triggers on `order_items`

Automatically recalculate the order's `total_amount` whenever an order item is inserted, updated, or deleted

Re-runnable script

`DROP TRIGGER IF EXISTS` is used before creating each trigger so that the script can be safely executed again

Changelog

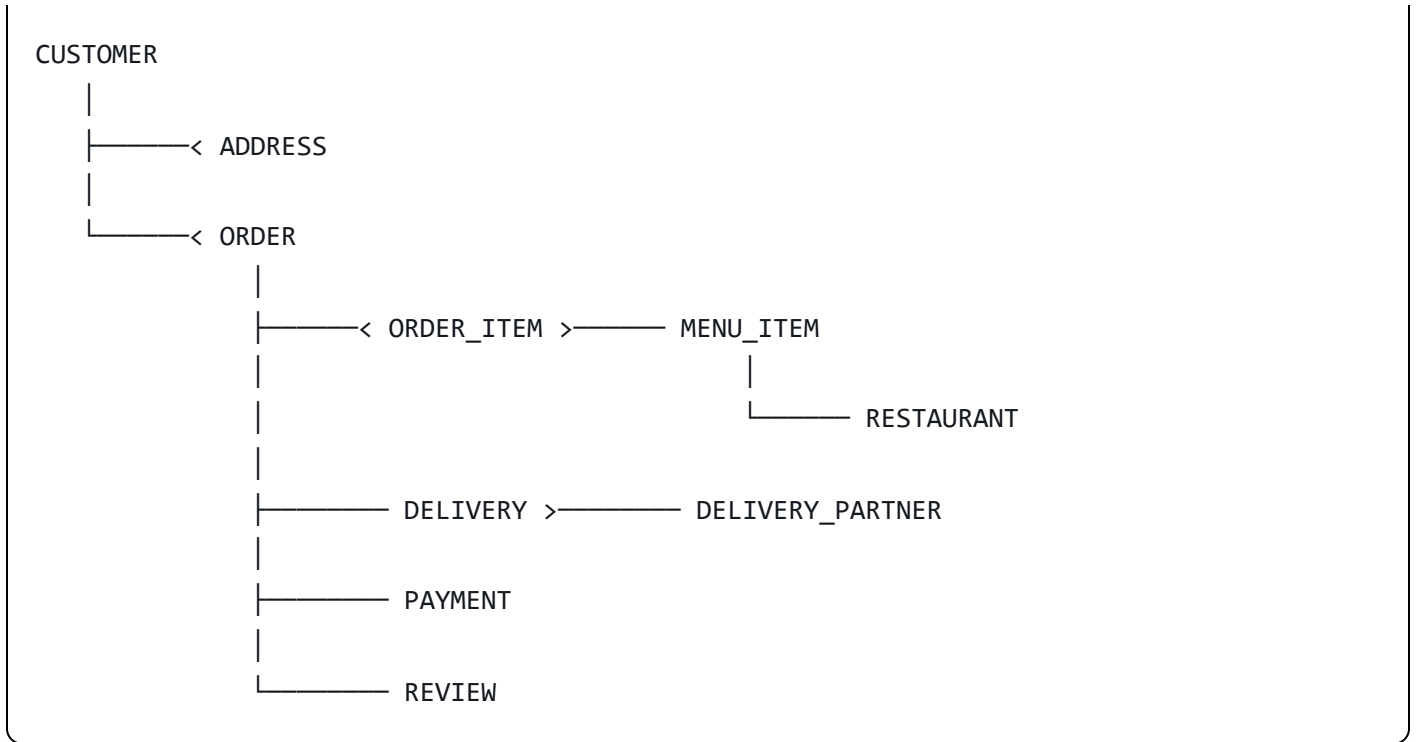
A changelog was added to document the modifications

[!IMPORTANT] The existing unique constraints were **not removed**; only trailing commas were added so the new `CHECK` constraints could follow them. These changes are **backward-compatible**: the teammates' `insert_tables.sql`, `queries.sql`, and `normalization_validation.sql` files continue to work without modification and produce the same results. They only need to pull the updated `create_tables.sql` when rebuilding the database from scratch.

Relationships

The major relationships are:

text



Cardinalities

Relationship	Cardinality
Customer → Address	1 : N
Customer → Order	1 : N
Restaurant → Menu Item	1 : N
Order ↔ Menu Item	M : N through order_items
Restaurant → Order	1 : N
Order → Delivery	0 : 1 in the current schema
Delivery Partner → Delivery	1 : N
Order → Payment	0 : 1 in the current schema
Order → Review	0 : 1 in the current schema

Normalization

Normalization is the process of organizing relational data to **reduce unnecessary duplication** and **avoid update, insertion and deletion anomalies**. FoodGo includes a separate

`normalization.sql` script that performs read-only demonstrations and verification.

It **does not use**: ❌ `CREATE` ❌ `ALTER` ❌ `DROP` ❌ `INSERT` ❌ `UPDATE` ❌ `DELETE`

The script therefore **does not modify** the supplied database.

Normal Forms at a Glance — Comparison

	 UNF	 1NF	 2NF	 3NF
Rule	No rules applied; repeating groups allowed	Atomic values; no repeating groups	Already 1NF + every non-key attribute depends on the whole key	Already 2NF + no non-key attribute depends on another non-key attribute (no transitive dependency)
Problem it fixes	—	Multi-valued / repeating item lists in one row	Partial dependency on part of a composite key	Transitive dependency between non-key attributes
FoodGo example	<code>Order 2 → [Penne Alfredo, Grilled Salmon Steak]</code>	One item = one row in <code>order_items</code>	<code>(order_id, item_id) → quantity, unit_price</code>	<code>order_id → address_id → customer_id</code>
Key used	None	<code>PRIMARY KEY (order_id, item_id)</code>	Composite key <code>(order_id, item_id)</code>	Every determinant is a key of its own relation
FoodGo status	Conceptual only (not a physical table)	✅ Demonstrated	✅ Demonstrated	⚠️ Analyzed; <code>orders</code> retains a transitive dependency
Verified by	—	N04	N05, N06, N07	N08 - N14

🗨️ Why 3NF Is the Optimal Target

🔗 Reason 💬 Explanation

🔪 Removes redundancy	Each fact (customer, address, restaurant, item, partner) is stored in exactly one place, so nothing is repeated across rows
🛡️ Prevents anomalies	Eliminates update, insertion and deletion anomalies; changing a customer's details never requires touching many order rows
🔗 Lossless reconstruction	Decomposed tables can be joined back to the original without losing or inventing rows (verified in N14)
📊 Dependency preservation	Business rules remain enforceable inside individual tables using keys and constraints
⚖️ Balanced design	Stronger forms (e.g., BCNF and above) add complexity with little extra benefit for this schema, while lower forms leave anomalies behind. 3NF gives the best trade-off between integrity and practical query performance

🔍 Stage-by-Stage Detail

🟥 Unnormalized Form — UNF

A conceptual unnormalized order could look like:

```
text

Order
├─ order_id
├─ customer
├─ restaurant
└─ items
    ├─ Item 1
    ├─ Item 2
    └─ Item 3
```

The problem is the **repeating group of items**. The project does not create this as a physical table; it is used only to explain the starting normalization problem.

🟠 First Normal Form — 1NF

A relation is in 1NF when attributes contain **atomic values** and **repeating groups are removed**.

FoodGo demonstrates this through the order-line query in `normalization.sql`.

Instead of:

text

Order 2 → [Penne Alfredo, Grilled Salmon Steak]

the order items appear as separate rows:

order_id	item_id	item_name
2	74	Penne Alfredo
2	77	Grilled Salmon Steak

Therefore: **One item = one row**. The `order_items` table uses `PRIMARY KEY (order_id, item_id)` to uniquely identify an order-item combination.

🟠 Second Normal Form — 2NF

2NF requires:

1. The relation is already in 1NF.
2. Non-key attributes depend on the **whole key**, not only part of a composite key.

This is particularly relevant to `order_items`. Its composite key is `(order_id, item_id)` and the dependency is:

text

`(order_id, item_id)`

↓

`quantity, unit_price`

The normalization script demonstrates this by separating:

- 📄 order-level facts,
- 🍷 menu-item facts,
- 🛒 order-item facts.

This avoids storing order-level information repeatedly on every item row.

🟢 Third Normal Form — 3NF Analysis

For the independent relations, the important dependencies include:

Relation	Functional Dependency
 customers	$\text{customer_id} \rightarrow \text{name, phone, email}$
 addresses	$\text{address_id} \rightarrow \text{customer_id, line1, area, city, zipcode}$
 restaurants	$\text{restaurant_id} \rightarrow \text{name, cuisine, city, is_active}$
 menu_items	$\text{item_id} \rightarrow \text{restaurant_id, name, price, is_veg}$
 delivery_partners	$\text{partner_id} \rightarrow \text{name, phone, vehicle_type}$
 order_items	$(\text{order_id, item_id}) \rightarrow \text{quantity, unit_price}$
 orders	$\text{order_id} \rightarrow \text{customer_id, restaurant_id, address_id, order_time, status, total_amount}$

[!WARNING] **Important** `orders` **dependency:** Under the project's stated business rule $\text{address_id} \rightarrow \text{customer_id}$ (because an address belongs to one customer), we get:

```
text
order_id
  ↓
address_id
  ↓
customer_id
```

This is a **transitive dependency**. Because of this, the current physical `orders` table is **not strict 3NF** under that stated dependency. The project deliberately does not hide this issue.

The normalization script:

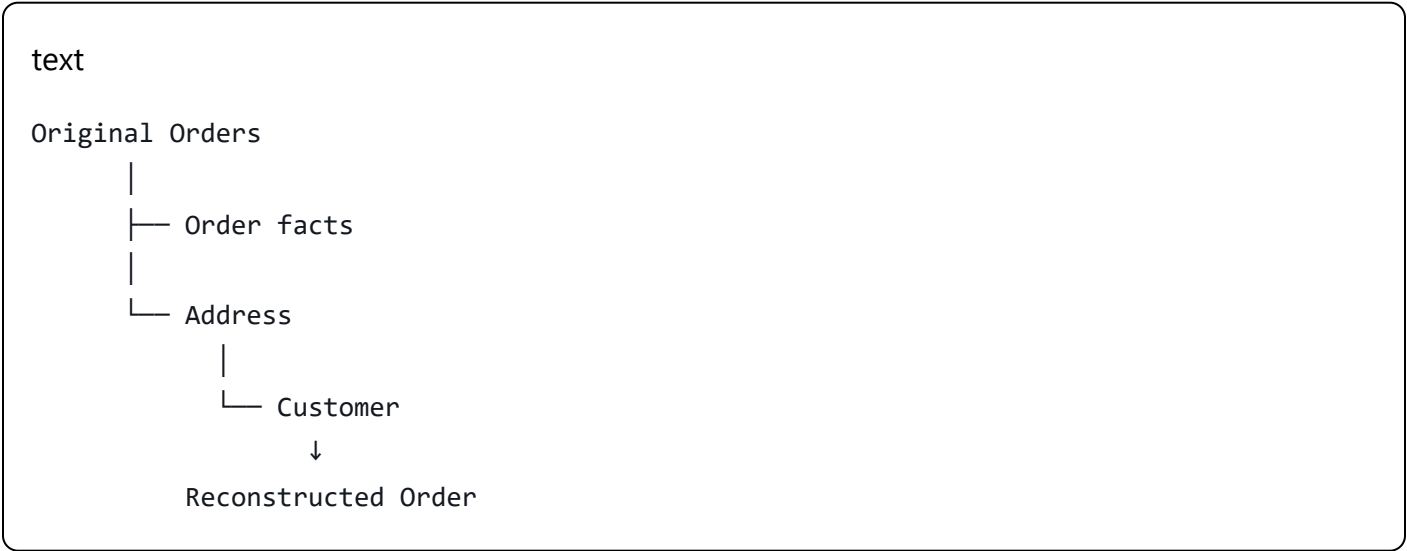
-  demonstrates the 3NF projection,
-  separates address and customer facts conceptually,
-  checks reconstruction,
-  verifies that the current data has no observed ownership mismatch,
-  preserves the original physical schema.

A possible future redesign could omit `orders.customer_id` and derive the customer through `addresses`, but that would require reviewing existing queries, application logic and business

rules.

 Lossless Reconstruction

Query **N14** checks whether the order information can be reconstructed after projecting out `customer_id`. Conceptually:



The verification compares: original order count, reconstructed order count, reconstructed header values.

Metric	Expected Result
<code>original_orders</code>	40
<code>reconstructed_orders</code>	40
<code>mismatched_headers</code>	0

This demonstrates lossless reconstruction for the tested data.

 Price Snapshot

`menu_items.price` represents the **current catalog price**. `order_items.unit_price` represents the **historical price** captured when the order was placed.

Source	Value
Current menu price (<code>menu_items.price</code>)	₹350
Historical order price (<code>order_items.unit_price</code>)	₹299

The order must retain ₹299 even if the restaurant later changes the catalog price to ₹350. Therefore `unit_price` is intentionally stored in `order_items`.

Total Verification

The normalization script compares:

```
text
Stored total    vs    SUM(quantity × unit_price)
```

Item	Calculation	Amount
Penne Alfredo	3 × 299	₹897
Grilled Salmon Steak	2 × 449	₹898
Total	897 + 898	₹1795

The verification query checks whether the stored order total matches the calculated total.

Integrity Constraints

 Constraint	 Purpose	 Examples
 Primary Keys	Uniquely identify records	<code>customer_id</code> , <code>restaurant_id</code> , <code>order_id</code> , <code>item_id</code>
 Composite Primary Key	Uniquely identify a combination	<code>(order_id, item_id)</code> in <code>order_items</code>
 Foreign Keys	Maintain relationships between tables	<code>orders.customer_id → customers.customer_id</code> <code>orders.restaurant_id → restaurants.restaurant_id</code> <code>order_items.order_id → orders.order_id</code> <code>order_items.item_id → menu_items.item_id</code>
 UNIQUE	Values that must not repeat	<code>customers.phone</code> , <code>customers.email</code> , <code>delivery_partners.phone</code> , <code>deliveries.order_id</code> , <code>payments.order_id</code> , <code>reviews.order_id</code>
 NOT NULL	Used where a value is required	Required columns across tables
 CHECK	Validate values	<code>menu_items.price > 0</code> , <code>order_items.quantity > 0</code> , <code>order_items.unit_price > 0</code> , ratings between 1 and 5
 DEFAULT	Automatic default values	<code>order_time = CURRENT_TIMESTAMP</code> , <code>status = 'placed'</code>
 Referential Actions	Maintain referential integrity	Appropriate <code>ON DELETE</code> and <code>ON UPDATE</code> actions such as <code>CASCADE</code> and <code>RESTRICT</code>

Sample Data

The sample-data script contains realistic FoodGo data covering:

 Customers	 Addresses	 Restaurants	 Menu items	 Delivery partners
 Orders	 Order items	 Deliveries	 Payments	 Reviews

The current sample-data source documents **347 total records** across the 10 tables.

The data is designed to be referentially consistent and sufficient to produce meaningful query results.

[!NOTE] Record counts can depend on the database snapshot used for execution. The normalization report and sample-data script should therefore be treated as **execution-specific evidence** rather than assumed to describe every future database state.

✅ Normalization Verification Queries

The `normalization.sql` file contains the following analysis stages:

Query	Purpose	Category
N01	Verify database environment	 Setup
N02	List existing base tables	 Setup
N03	Inspect primary and unique constraints	 Setup
N04	Demonstrate 1NF order-line representation	 1NF
N05	Display order-level facts for 2NF	 2NF
N06	Display menu-item facts for 2NF	 2NF
N07	Demonstrate complete composite-key dependency	 2NF
N08	Verify address ownership for a selected order	 3NF
N09	Check address ownership across all orders	 3NF
N10	Show the 3NF order projection	 3NF
N11	Show address facts	 3NF
N12	Show customer facts	 3NF
N13	Show restaurant facts	 3NF
N14	Verify lossless reconstruction	 Lossless
N15	Compare current and historical prices	 Consistency
N16	Compare stored and calculated total	 Consistency
N17	Verify totals across all orders	 Consistency
N18	Check order/menu-item restaurant consistency	 Consistency
N19	Show final database inventory	 Summary

Business Questions

The project addresses the core questions from the DBMS project specification and additional domain questions.

#	Business Question	SQL Concepts
Q1	 What is the most ordered food item?	<code>JOIN</code> , <code>SUM</code> , <code>GROUP BY</code>
Q2	 Which restaurant has the highest revenue?	Multi-table <code>JOIN</code> , <code>SUM</code> , ranking
Q3	 What is the average order value?	CTE/subquery, <code>AVG</code>
Q4	 Who are the top customers by spending?	Aggregation, <code>LIMIT</code>
Q5	 What is the average delivery time?	Time-difference functions, <code>AVG</code>
Q6	 Which restaurants have high ratings?	<code>AVG</code> , <code>HAVING</code>
Q7	 How many orders were cancelled?	<code>COUNT</code> , <code>CASE</code> , filtering
Q8	 What are the peak ordering hours?	<code>HOUR()</code> , <code>GROUP BY</code>
Q9	 How does delivery-partner performance compare?	<code>JOIN</code> , <code>AVG</code> , aggregation/ranking
Q10	 What is the revenue by cuisine?	Multi-level <code>GROUP BY</code>
Q11	 Which customers order from multiple restaurants?	<code>COUNT(DISTINCT ...)</code> , <code>HAVING</code>
Q12	 What revenue is associated with cancelled orders?	Conditional aggregation

💡 Key Design Decisions

1 Price Snapshot

`order_items.unit_price` is stored separately from `menu_items.price`.

Column	Meaning
<code>menu_items.price</code>	Current catalog price
<code>order_items.unit_price</code>	Price recorded for that historical order

Reason: This preserves historical order accuracy.

2 Composite Key in `order_items`

```
sql
```

PRIMARY KEY (order_id, item_id)

An item can appear in many orders and an order can contain many items. The composite key uniquely identifies each order-item combination.

3 At-Most-One Delivery, Payment and Review

Unique constraints on (order_id) in (deliveries), (payments) and (reviews) ensure that an order cannot have multiple rows in each of those tables. The current physical schema enforces **at most one**, not necessarily **exactly one**.

4 Stored (total_amount)

The (orders) table stores (total_amount). The normalization verification script separately calculates $\text{SUM}(\text{quantity} \times \text{unit_price})$ and compares it with the stored total. This provides a direct consistency check against update/calculation errors.

5 Referential Integrity

Foreign keys and referential actions are used to prevent orphaned records and maintain relationships between entities.

6 Read-Only Normalization Verification

The normalization script does not alter the existing database. This was intentional because the objective is to demonstrate and verify normalization against the supplied schema and data, while preserving the project database.

🗨️ SQL Concepts Demonstrated

🏷️ Category

🧰 Concepts

Language

DDL, DML

Querying

SELECT, WHERE, ORDER BY, GROUP BY, HAVING

Joins &
Subqueries

JOIN, LEFT JOIN, EXISTS, Subqueries, CTEs

Aggregates

SUM(), COUNT(), AVG()

Logic & Sets

CASE, UNION ALL

Functions

Date/time functions

Constraints

Primary keys, Foreign keys, Unique constraints, Composite keys, Check constraints, Default values, Referential integrity

Theory

Functional dependencies, 1NF, 2NF, 3NF analysis, Lossless reconstruction

🚀 How to Run the Project

✅ Prerequisites

Requirement

Details

🗄️ MySQL Server

Version **8.x**

💻 Client

MySQL command-line client **or** MySQL Workbench

🔧 Git (*optional*)

Needed only for the Git/command-prompt method

📋 Execution Order (Important!)

[!IMPORTANT] The scripts **must be run in this exact order**, because each step depends on the previous one.

1 create_tables.sql
Schema

2 insert_data.sql
Sample Data

3 queries.sql
Business Queries

4 normalization.sql
Verification

5 Verify
SHOW TABLES

Step	Script	What it does
1	<code>schema/create_tables.sql</code>	Creates the <code>foodgo</code> database and its tables
2	<code>data/insert_data.sql</code>	Populates the tables with sample data
3	<code>queries/queries.sql</code>	Executes the business-analysis queries
4	<code>normalization/normalization.sql</code>	Executes the read-only normalization and consistency checks
5	<i>(manual)</i>	Verify the database using <code>USE foodgo; SHOW TABLES;</code>

Option A — Command Prompt / Terminal + Git

Step 0 — Clone the repository

```
bash

git clone https://github.com/<your-username>/FoodGo.git
cd FoodGo
```

Method A1 — Run each script directly from the shell

```
bash

# 1 Create the database and tables
mysql -u root -p < schema/create_tables.sql

# 2 Insert sample data
mysql -u root -p < data/insert_data.sql

# 3 Run business queries
mysql -u root -p < queries/queries.sql

# 4 Run normalization verification (read-only)
mysql -u root -p < normalization/normalization.sql
```

Method A2 — Run from inside the MySQL prompt

```
bash
```

```
mysql -u root -p
```

```
sql
```

```
-- 1 Create the database and tables
SOURCE schema/create_tables.sql;

-- 2 Insert sample data
SOURCE data/insert_data.sql;

-- 3 Run business queries
SOURCE queries/queries.sql;

-- 4 Run normalization verification
SOURCE normalization/normalization.sql;

-- 5 Verify the database
USE foodgo;
SHOW TABLES;
```

[!TIP] Run `mysql` from the **repository root folder** so the relative paths (`schema/...`, `data/...`) resolve correctly. On Windows, use forward slashes in `SOURCE` paths (for example `SOURCE C:/Projects/FoodGo/schema/create_tables.sql;`).

Pulling later updates

If a teammate updates the schema, pull the latest files and rebuild:

```
bash

git pull origin main
mysql -u root -p < schema/create_tables.sql
mysql -u root -p < data/insert_data.sql
```

Option B — MySQL Workbench

Step Action

- 1 Open **MySQL Workbench** and connect to your local MySQL server.
- 2 Go to **File → Open SQL Script...** and select `schema/create_tables.sql`.
- 3 Click the  **Execute** button (or press `Ctrl + Shift + Enter`) to create the database and tables.
- 4 Open `data/insert_data.sql` the same way and execute it to load the sample data.
- 5 Open `queries/queries.sql` and execute it to view the business-analysis results.
- 6 Open `normalization/normalization.sql` and execute it to run the read-only checks.
- 7 In the **Schemas** panel, click  refresh, then expand `foodgo → Tables` to confirm all **10 tables** exist.

[!NOTE] If a script contains several queries, Workbench shows each result in its own **Result Grid** tab. To run a single query, highlight it and press `Ctrl + Enter`.

Quick Verification

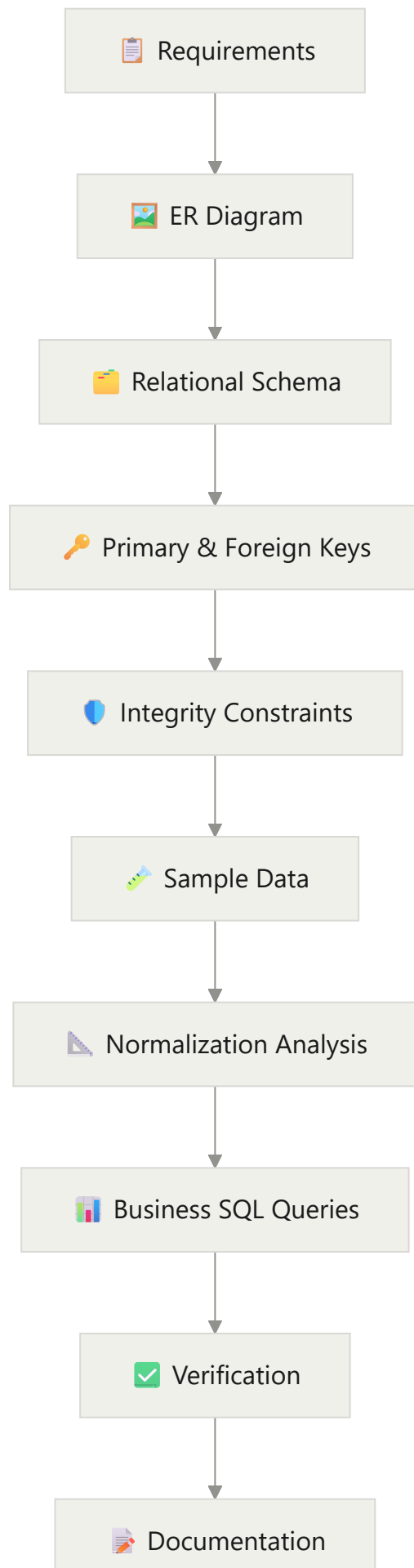
```
sql
USE foodgo;
SHOW TABLES;                -- should list the 10 core tables
SELECT COUNT(*) FROM orders;  -- sample data contains 40 orders
```

Expected result from normalization query **N14**:

original_orders	reconstructed_orders	mismatched_headers
40	40	0

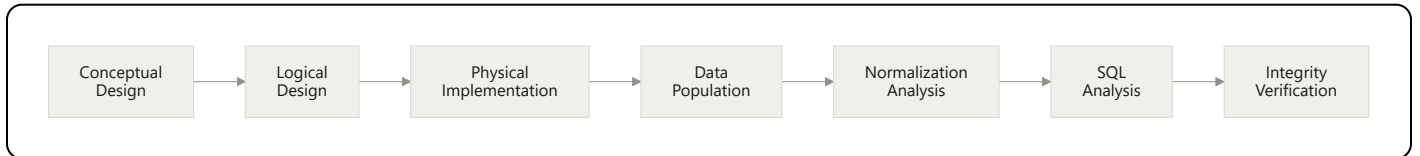
Project Workflow

The overall project follows:



❏ Conclusion

FoodGo demonstrates how a real-world food-delivery system can be represented using a relational database. The project covers the complete DBMS workflow:



The database provides structured relationships between customers, addresses, restaurants, menu items, orders, order items, delivery partners, deliveries, payments and reviews.

The dedicated normalization analysis makes the project's assumptions and functional dependencies explicit. It demonstrates 1NF and 2NF, evaluates the 3NF condition of the current schema, verifies lossless reconstruction and performs additional consistency checks without modifying the database.

 **Team 5 — FoodGo Restaurant, Order & Delivery Management System**

DBMS Course Project